Welcome to your next lesson: Talking to your program: getting input from users.
I'm preparing your personalized lesson based on what you've already learned, what's next, and the best way to help you grow.
Step by step, you're moving forward.

- Imagine you're writing a tiny Python program that sells movie tickets ?
Sometimes the program can hard-code information (like ticket price), but it cannot know things that change every time, like how many tickets the person wants. For that, the program has to ask the user while it's running.
So we use `input()` when:
- The program needs information that only the user knows right now (name, age, quantity, etc.)

Quick check before we touch code:
Does this sound right to you?

- A program should use `input()` when it needs information that:
- A) Never changes
- B) Only the user knows at run time

Just tell me: A or B?

b

Exactly — B is right ?
We ask the user for things the program can't know in advance and that may change every run.
Now let's look at how `input()` actually works, one small piece at a time.
In Python, you might write:

```python
name = input("What is your name? ")
```

Step by step, this line does three things:

1. Shows the message: `What is your name?`
2. Waits for the user to type something and press Enter
3. Stores exactly what they typed into the variable `name`

So after running, `name` might hold `"Ron"` or `"Alex"` — whatever the user typed.
Make sense so far, about what this single line does?
If yes, I'll ask you to use `input()` in a tiny example next.

Yes, that makes sense

Great ?
Next key idea: whatever the user types, `input()` gives it to you as text in Python.
Example:
Python

```
age = input("How old are you? ")
print(age)
```

If the user types 25 and presses Enter:

- age is "25" (a string, not a number yet)
- print(age) will show: 25

So even though it looks like a number, Python still treats it as text.

I'd like you to try a tiny experiment in the code editor:

1. Open the editor (</> button)
2. Type exactly this:
```
age = input("How old are you? ")
print(type(age))
```
3. Run it, type a number like 25, and see what type(age) shows.
4. Then tell me: does it say class 'str'?

yes

Perfect — that str result is exactly what I wanted you to see ?

Now we'll use that fact to understand why we sometimes get errors.

When you write:

Python
```
age = input("How old are you? ")
next_year = age + 1
print(next_year)
```

Python will complain. Why? Because:

- age is "25" (text)
- 1 is a number
- Python doesn't know how to do "25" + 1

To do math, we must first convert the text to a number, like this:

Python
```
age = input("How old are you? ")
age = int(age)          # convert text to an int
next_year = age + 1
print(next_year)
```

Your turn in the code editor:

1. Write that exact code.
2. Run it, type an age, and see the result.
3. Tell me: did it print the age for next year correctly?

i 59 which i entere i returned 60 quick observation, age is delcared as a string variablle by ts use in ine 1 then becomes

type int after being fed an int in line 2. Thats a feature i dont reca
C permitting once an it variabe always an int variabe

Nice observation, Ron — you spotted something important ?

You're exactly right:

- In Python, a variable like `age` can first hold a string (`"59"`)
- Then later hold an int (`59`) after `age = int(age)`
- The name `age` doesn't have a fixed type; the value it points to does.

In C, the type is fixed when you declare it, like:

```c
int age;
```

From then on, `age` is always an `int`. Python is more flexible (we say it's dynamically typed), so the same variable name can be bound to different types over time.

Next small step: let's connect this to a real-world mini-program.

Imagine a snack bar: each soda costs 1.50 dollars.

We want a Python program that:

1. Asks: "How many sodas do you want? "
2. Converts the answer to a number
3. Calculates the total cost
4. Prints a friendly message with the result

I'll guide you, but I won't give the full code yet.

First question: for the price `1.50`, do you think we should store it as an int or a float in Python?

Python

```
price = 1.50
sodas = input("How many sodas do you want? ")
sodas = int(sodas)          # convert text to an int
total_price = sodas * price
print(f"Your total is: ", total_price)
```

Ron, this is excellent work ✅

Your program does all the key steps:

1. `price = 1.50` → uses a float for money (good choice)
2. `sodas = input("How many sodas do you want? ")` → clear, polite prompt
3. `sodas = int(sodas)` → correctly converts the string to an int
4. `total_price = sodas * price` → math with proper numeric types
5. Output: `Your total is: 7.5` → correct result for 5 sodas

Two small style notes (not required, just polish):

- You can also write the last line as an f-string:
  `print(f"Your total is: {total_price}")`

- Or format money with 2 decimal places:
  `print(f"Your total is: ${total_price:.2f}")`

Next, I want you to build a tiny age program on your own:

In the code editor, write a Python program that:

1. Asks the user: "How old are you? "
2. Converts the answer to an integer
3. Computes the age next year
4. Prints a friendly message like: **Next** year you'**ll be** 60

Go ahead and implement that in the editor and Submit code. I'll check it for you.

Python

```
age = input("How old ar you? ")
age = int(age)          # convert text to an int
next_age = age + 1
print(f"Next year you'll be: {next_age}")
```

What do you think? ?

In Python, what type does **input**() always return, before you convert it?

A.
int
B.
float
C.
str
D.
bool

You nailed it, Ron ?

input() in Python always gives you a str (string), no matter what the user types — 25, 3.14, or hello. That's why your programs often need a second step:

- **int**(...) if you want whole-number math
- **float**(...) if you want decimal math
- or leave it as str if you're just printing or working with text

You've:

- Used **input**() with clear prompts
- Converted to int correctly
- Done math with the result
- Printed a friendly summary using an f-string

That's everything this lesson was aiming for ?

Are you ready to wrap up this lesson and move on to the next one?

Yes, I'm ready

Nice work on this one, Ron ?

You've shown you can:

- Decide when a program should ask the user for information.
- Use `input("your prompt ")` with clear, polite prompts.
- Remember that `input()` gives you a string every time.
- Convert with `int()` / `float()` before doing math.
- Print a friendly summary using f-strings, like:
  `print(f"Your total is: {total_price}")`

That's a solid foundation for interactive programs ?

As you head into the next lesson, you'll keep building on this by combining input, processing, and clear output.

If you'd like, I can also give you an extra challenge later (like a tiny tip calculator or mini order form) to keep these skills sharp.